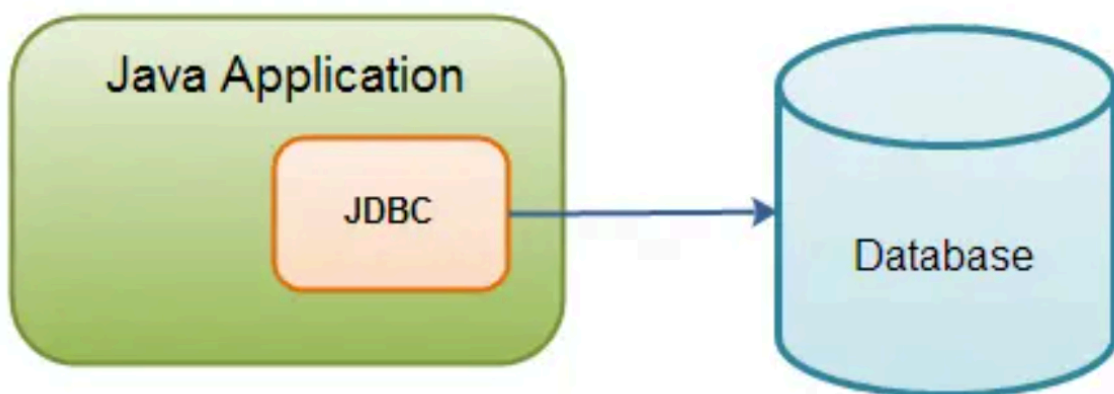


JDBC (Java Database Connectivity)



JDBC is an API in Java used to connect Java applications with databases



What is API?

API = Application Programming Interface

Simple meaning:

API is a collection of: Interfaces, Classes, Methods, Rules that **allows** two softwares to communicate.

Example

Customer → Waiter(API) → Kitchen

You don't **directly go to kitchen**. You talk through waiter.

Similarly:

Java **Program** → JDBC **API** → Database. **JDBC** acts **like** waiter.

Why JDBC Needed?

Database vendors are different: MySQL, Oracle, PostgreSQL, SQL Server
Each database has different internal implementation.
Java needed ONE COMMON WAY to communicate with all databases.
So Sun Microsystems created JDBC API.

JDBC is NOT:

SOFTWARE

Database

Driver

Server

JDBC is a set of interfaces/classes inside Java.

So, JDBC is an API that helps applications to communicate with databases. It allows Java programs to connect to a database, run queries, retrieve and manipulate data. Because of JDBC, Java applications can easily work with different relational databases like MySQL, Oracle, PostgreSQL and more.

Interfaces of JDBC API :

Class/Interface	Description
<u>DriverManager</u>	This class manages a list of database drivers. Matches connection requests from the Java application with the proper database driver using communication sub protocol. The first driver that recognizes a certain sub-protocol under JDBC will be used to establish a database Connection.
Driver	This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. In stead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects.
Connection	This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
Statement	You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
<u>ResultSet</u>	This class retrieves data from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
<u>SQLException</u>	This class handles any errors that occur in a database application.

JDBC steps

1. Load a JDBC driver.
2. Create connections.
3. Connect to the data source.
4. Execute SQL statement(s).
5. Map the results to data structures.

Load a JDBC driver

JDBC Driver : Driver is actual software/library that understands database language.

For MySql:

```
Class.forName ("com.mysql.cj.jdbc.Driver").newInstance();
```

For Oracle:

```
Class.forName("oracle.jdbc.driver.OracleDriver").newInstance();
```

For MS SQL Server :

```
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver").newInstance();
```

To connect your Java application to a database, you first initialize the driver and register it with the **DriverManager**. This allows the application to communicate with your specific SQL or NoSQL database.

Common syntax: `Class.forName("com.mysql.cj.jdbc.Driver");`

What Does Class.forName() Do?

Does it create object?

Not directly.

It:

Loads **class** into JVM memory

Executes **static** block

Internally Driver Class Looks Like

public class Driver

implements java.sql.Driver {

static {

 DriverManager.registerDriver(

new Driver()

);

 }

}

So When This Executes

Class.forName("com.mysql.cj.jdbc.Driver");

JVM:

1. Loads **Driver class**

2. Static block runs automatically
 3. **Driver** registers itself
- THIS IS CALLED **Loading** + Registering Driver

Create Connections

DriverManager in JDBC : **DriverManager** is one of the MOST IMPORTANT classes in JDBC that manages JDBC drivers, establishes database connections, selects appropriate driver.

Java application does NOT know MySQL internal communication, Oracle protocol, PostgreSQL protocol and different databases have different drivers so DriverManager acts as middle layer.

Syn: `DriverManager.getConnection(url,user,password);`

Example : `Connection con =`

```
    DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/test",  
        "root",  
        "root");
```

Create Statement

Statement is an **interface** used to send SQL **queries** to **database**, **execute SQL statements**, receive results.

Syntax: `Statement st = con.createStatement();`

`con.createStatement();`

executes internally: Connection **object** talks to DB session and **Statement** object created.

```
public class Demo {  
    public static void main(String[] args)  
    throws Exception {  
        // Load Driver  
        Class.forName(  

```

```

"com.mysql.cj.jdbc.Driver");
// Establish Connection
Connection con =
DriverManager.getConnection(
"jdbc:mysql://localhost:3306/test",
"root",
"root");
// Create Statement
Statement st =
con.createStatement();
System.out.println(
"Statement Created");
con.close();
}
}

```

Statement Method :

Method		Purpose
executeQuery()	:	SELECT
executeUpdate()	:	INSERT UPDATE DELETE
execute()	:	Any SQL
addBatch()	:	Batch processing

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;
public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(

```

```

        "jdbc:mysql://localhost:3306/test",
        "root",
        "root"
    );
    Statement st = con.createStatement();
    // insert
    int a = st.executeUpdate(
        "insert into student values(1,'Rahul')"
    );
    System.out.println("Inserted : " + a);
    // update
    int b = st.executeUpdate(
        "update student set name='Vivek' where id=1"
    );
    System.out.println("Updated : " + b);
    // delete
    int c = st.executeUpdate(
        "delete from student where id=1"
    );
    System.out.println("Deleted : " + c);
    // inserting again for select
    st.executeUpdate(
        "insert into student values(1,'Aman')"
    );
    st.executeUpdate(
        "insert into student values(2,'Karan')"
    );
    // select using executeQuery
    ResultSet rs = st.executeQuery(
        "select * from student"
    );
    System.out.println("\nStudent Data");
    while(rs.next()) {

```

```

        System.out.println(
            rs.getInt(1) + " " +
            rs.getString(2)
        );
    }
    // execute()
    boolean check = st.execute(
        "select * from student"
    );
    if(check) {
        ResultSet r = st.getResultSet();
        System.out.println("\nData using execute()");
        while(r.next()) {
            System.out.println(
                r.getInt(1) + " " +
                r.getString(2)
            );
        }
    }
    // batch
    st.addBatch(
        "insert into student values(3,'Simran')"
    );
    st.addBatch(
        "insert into student values(4,'Mohit')"
    );
    st.addBatch(
        "update student set name='Akash' where id=2"
    );
    int arr[] = st.executeBatch();
    System.out.println("\nBatch Executed");
    for(int i : arr) {
        System.out.println(i);
    }
}

```

```

    }
    // final table
    ResultSet rs2 = st.executeQuery(
        "select * from student"
    );
    System.out.println("\nFinal Data");
    while(rs2.next()) {
        System.out.println(
            rs2.getInt(1) + " " +
            rs2.getString(2)
        );
    }
    con.close();
}
catch (Exception e) {
    System.out.println(e);
}
}
}

```

PreparedStatement in JDBC

PreparedStatement is a child interface of Statement used for dynamic queries better performance prevent SQL Injection.

```
String q ="insert into student values(" + id + "," + name + ")";
```

Problem with Statement Query it compiled every time, **SQL Injection possible, Not secure.**

SQL Injection: SQL injection (SQLi) is a critical web security vulnerability that allows attackers to interfere with database queries, often enabling them to view, modify, or delete sensitive data. **It occurs when untrusted user input is directly concatenated into SQL commands.**

Simple Meaning instead of giving normal input user gives SQL code as input and database executes it eg:

```
select * from users where username='Rahul' and password='123';
```


Problematic Java Code

```
String username = "Vivek";
String password = "123";
String q =
"select * from users where username="
+ username +
" and password="
+ password + "";
```

Query Becomes

```
select * from users
where username='Vivek'
and password='123' ;           Works fine.
```

Query with SQL injection: Suppose hacker enters Password :
' OR '1'='1' ; So query becomes

```
select * from users where username='Rahul' and password=" OR '1'='1' ;
```

'1'='1' is ALWAYS TRUE so database returns login success without correct password this is called SQL Injection Attack.

But PreparedStatement **prevents SQL Injection** because it uses placeholders (?) and treats user input as normal data instead of executable SQL code, so even if a hacker enters input like ' OR '1'='1', the database considers it a plain string and does not execute it as part of the SQL query.

In PreparedStatement query compiles only ONCE later only values change and PreparedStatement is Faster, Secure, Reusable.

Syntax:

```
PreparedStatement ps = con.prepareStatement("insert into student values(?,?)");
```

? Called: Placeholder / Parameter Marker value will come later means value will replace ? later.

Example:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
public class Demo {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
```

```

        "jdbc:mysql://localhost:3306/test",
        "root",
        "root"
    );
    String q = "insert into student values(?,?)";
    PreparedStatement ps =
        con.prepareStatement(q);
    ps.setInt(1, 101);
    ps.setString(2, "Rahul");
    int rows = ps.executeUpdate();
    System.out.println(
        rows + " row inserted"
    );
    ps.close();
    con.close();
}
catch (Exception e) {
    System.out.println(e);
}
}
}

```

ResultSet

ResultSet is an object that stores rows returned from database data from SELECT query.

Suppose **database** table:

id	name
1	Aman
2	Rahul

After SELECT query:

```
ResultSet rs =st.executeQuery("select * from student");
```

that table data comes into: rs

next() Method : Initially cursor is before first row NOT on first row and next()

Method Moves cursor forward rs.next(). next() returns boolean like Return **true**

Meaning **row** exists and **false** if no **more** rows exists.

```
while(rs.next()) {  
    System.out.println(  
        rs.getInt(1) + " " +  
        rs.getString(2)  
    );  
}
```

Complete Example

```
import java.sql.*;  
public class Demo {  
    public static void main(String[] args)  
    throws Exception {  
        Class.forName(  
            "com.mysql.cj.jdbc.Driver");  
        Connection con =  
            DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/test",  
                "root",  
                "root");  
        Statement st =  
            con.createStatement();  
        ResultSet rs =  
            st.executeQuery(  
                "select * from student");  
        while(rs.next()) {  
            System.out.println(  
                rs.getInt(1)+" "+
```

```

        rs.getString(2));
    }
    con.close();
}
}

```

Data Fetching Methods of Result Set

Method	Purpose
getInt()	int value
getString()	String value
getDouble()	double value
getBoolean()	boolean value
getDate()	Date value

Summary

JDBC is the bridge between Java applications and databases. It provides a standard API through which Java can communicate with different databases like MySQL, Oracle, PostgreSQL, etc.

In JDBC we learned:

- Loading and registering **driver**
- Establishing **connection**
- DriverManager
- Statement
- executeQuery()
- executeUpdate()
- execute()
- Batch processing
- ResultSet and cursor

- CRUD operations
- Internal working of JDBC

Mini Project's

Bank Management System

Features:

- Create Account
- Deposit
- Withdraw
- Transfer Money
- Mini Statement

Employee Management System

Use:

JDBC

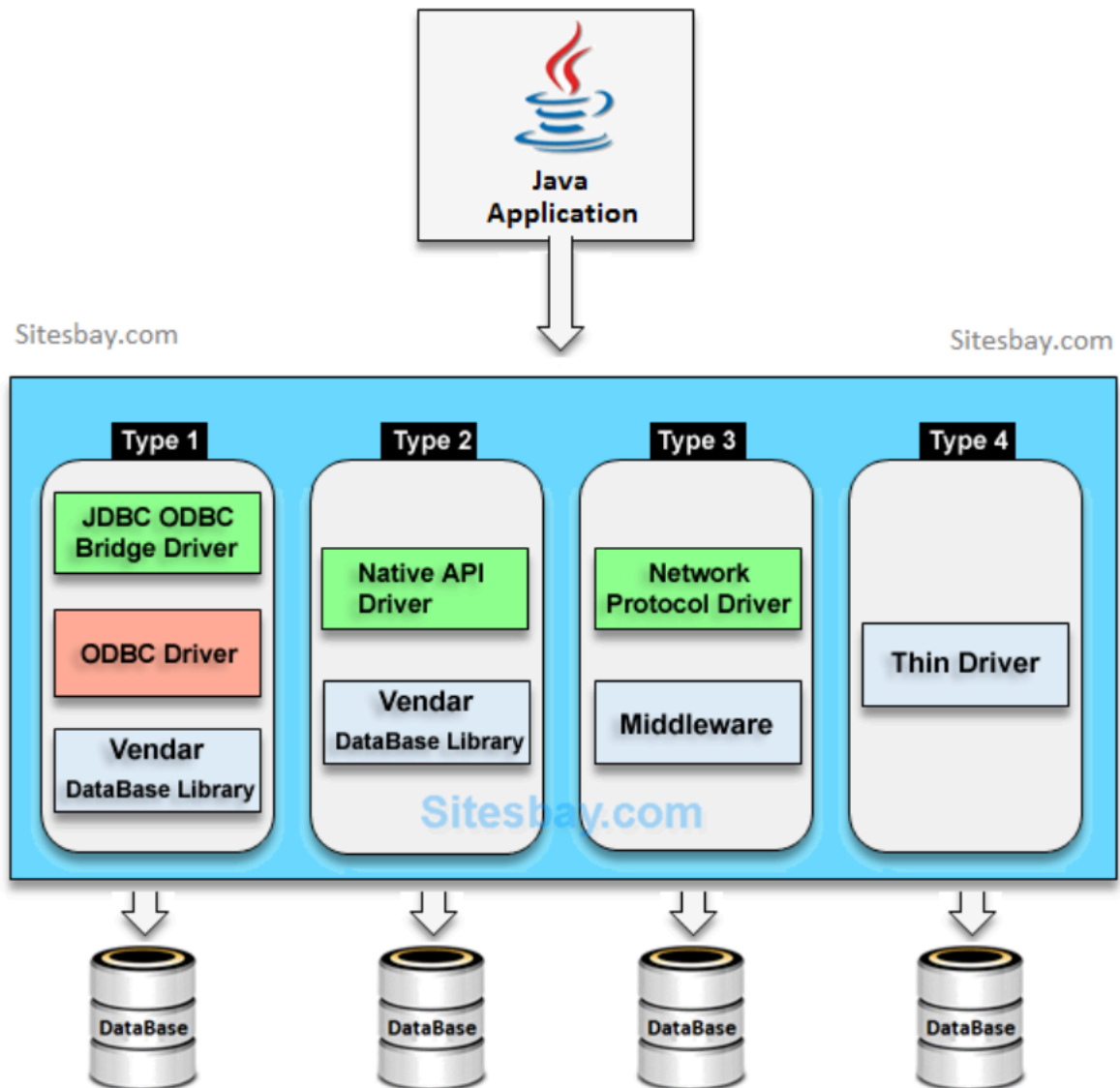
DAO Pattern

Batch Processing

Operations:

- add employee
- bulk insert employees
- update salary
- search employee
- delete employee

JDBC Drivers



JDBC drivers are software components that help Java applications communicate with databases.

There are 4 Types of JDBC Drivers :

1. Type 1 Driver : JDBC-ODBC Bridge Driver

ODBC (Open Database Connectivity):

ODBC is an API developed by Microsoft Used for connecting applications with databases. Earlier every database had different communication system so Microsoft created one common standard called ODBC

Example:

Oracle different

MySQL different

SQL Server different

So, ODBC acts like Universal Translator between application and database

JDBC-ODBC Bridge : Java initially could not directly talk to databases so Sun Microsystems created JDBC-ODBC Bridge Driver means JDBC calls converted into ODBC calls then ODBC communicates with DB.

Problem with Type 1 Driver : It is Platform Dependent and ODBC mainly Windows-based with that ODBC Installation Needed in Every machine so it is removed in Java 8 because it was outdated, slow, platform dependent, Type 4 drivers became better.

2. Type 2 Driver : Native API Driver

Type 2 driver is the second type of JDBC driver it uses Native Database Libraries written in C C++. Type 2 Introduced because Type 1 was too slow and have too many layers so Type 2 removed ODBC bridge and directly used native database APIs.

Native Library : Database vendors provide libraries written in C / C++ example .dll files in Windows, .so files in Linux. These libraries directly communicate with database means Java does not directly talk to database. Java calls native code and native code talks to database.

Problem with Type 2 Driver: It is platform dependent because it uses OS-specific native libraries (.dll/.so files), and native library installation is required on every machine, making deployment and maintenance difficult along with possible security risks, so Type 4 drivers became more preferred.

3. Type 3 Driver : Network Protocol Driver

Type 3 driver is the third type of JDBC driver. It uses a Middleware Server between Java application and database. Type 3 was introduced to solve the problems of Type 1 and Type 2 drivers like platform dependency and native library installation.

Middleware Server : Middleware server acts like a middle communication layer between Java application and database. Java application sends JDBC calls to the middleware server, then middleware converts those calls into database-specific protocol and communicates with the database. This means Java does not directly talk to the database.

Problem with Type 3 Driver: It requires a separate middleware server for communication which increases network overhead, complexity, and maintenance. Due to the extra middle layer performance becomes slower, so Type 4 drivers became more preferred because they directly communicate with the database.

4. Type 4 Driver : Thin Driver / Pure Java Driver

Type 4 driver is the fourth and most used type of JDBC driver. It is called Thin Driver because it directly communicates with the database without using ODBC, native libraries, or middleware server. It is completely written in Java.

Pure Java Driver : Type 4 driver converts JDBC calls directly into database-specific protocol and communicates with the database using socket connection. This means Java application directly talks to the database without any middle layer.

Example :

- MySQL Driver → `com.mysql.cj.jdbc.Driver`
- Oracle Driver → `oracle.jdbc.driver.OracleDriver`

Advantages of Type 4 Driver :

- Fastest JDBC driver
- Platform independent
- No native library required
- No middleware server required
- Easy to deploy and maintain
- Most preferred in real projects

Problem with Type 4 Driver: It is database dependent because different databases require different Type 4 drivers, but still it became industry standard because of high performance, simplicity, portability, and direct communication with database.

Short Description of Jdbc drivers

- Type 1 driver are database independent and platform dependent.
- Type 2 driver are database dependent and platform dependent.
- Type 3 driver are database independent and platform independent.
- Type 4 driver are database dependent and platform independent.

